No speedup when summing uint16 vs uint64 arrays with NumPy?

Asked 1 year, 11 months ago Modified 1 year, 11 months ago Viewed 1k times

6

I have to do a large number of operations (additions) on relatively small integers, and I started considering which datatype would give the best performance on a 64 bit machine.

I was convinced that adding together 4 `uint16` would take the same time as one `uint64`, since the ALU could make 4 `uint16` additions using only 1 `uint64` adder. (Carry propagation means this doesn't work that easily for a single 64-bit adder, but this is how integer SIMD instructions work.)

Apparently this is not the case:

```
In [3]: data = np.random.rand(10000)

In [4]: int16 = data.astype(np.uint16)

In [5]: int64 = data.astype(np.uint64)

In [6]: int32 = data.astype(np.uint32)

In [7]: float32 = data.astype(np.float32)

In [8]: float64 = data.astype(np.float64)

In [9]: %timeit int16.sum()
13.4 µs ± 43.3 ns per loop (mean ± std. dev. of 7 runs, 100000 loops each)

In [10]: %timeit int32.sum()
13.9 µs ± 347 ns per loop (mean ± std. dev. of 7 runs, 100000 loops each)

In [11]: %timeit int64.sum()
9.33 µs ± 47.8 ns per loop (mean ± std. dev. of 7 runs, 100000 loops each)

In [12]: %timeit float32.sum()
5.79 µs ± 6.51 ns per loop (mean ± std. dev. of 7 runs, 100000 loops each)

In [13]: %timeit float64.sum()
6 µs ± 3.54 ns per loop (mean ± std. dev. of 7 runs, 100000 loops each)
```

All int operations take the same time (more than the float ops) with no speedup. Is this due the C implementation of numpy not being fully optimized or is there some hardware limitation that I am not aware of?

python numpy performance compiler-optimization simd

Share Improve this question Follow

edited Nov 27, 2021 at 16:02
Jérôme Richard
42.9k 6 30 62

asked Nov 27, 2021 at 10:39
pnjun
111 8

1 *with no optimization* makes all your performance tests void. It makes no sense to do ant performance tests without optimizations – 0_____ Nov 27, 2021 at 10:44

1 x86-64 has SIMD integer add with 8, 16, 32, or 64-bit element width, and those instructions all have the same performance (felixcloutier.com/x86/paddb:paddw:paddd:paddq). (@mkrieger1: that's what SIMD is for. We don't normally call it "the ALU", though, partly because CPUs with SIMD normally also have multiple execution units). A well-optimized asm loop (generated by a C compiler) should run at the same speed in bytes per cycle (e.g. 2x 32-byte vectors loaded and summed per clock cycle, if data is hot in L1d cache), whether that's 2x 16x uint16 or 2x 4x uint64. – Peter Cordes Nov 27, 2021 at 10:51

Join Stack Overflow to find the best answer to your technical question, help others answer theirs.

Sign up with email Sign up with Google Sign up with GitHub Sign up with Facebook

3 NumPy does have some SIMD code (see for example github.com/numpy/numpy/tree/main/numpy/core/src/common/simd and github.com/numpy/numpy/tree/main/numpy/core/src/_simd), but I know very little about it. However, I believe operations on two separate, C-contiguous arrays (such as `int16 + int16`) get more development focus than reduction operations (like `int16.sum()`). `int16 + int16` may trigger a SIMD code path if `int16.sum()` doesn't. – user2357112 Nov 27, 2021 at 11:06

1 Answer

Sorted by: Highest score (default)

11

TL;DR: I made an experimental analysis on Numpy 1.21.1. Experimental results show that `np.sum` **does NOT (really) make use of SIMD instructions**: no SIMD instruction are used for integers, and scalar SIMD instructions are used for floating-point numbers! Moreover, Numpy **converts the integers to 64-bits values** for smaller integer types by default so to avoid *overflows*!

Note that this may not reflect all Numpy versions since there is an ongoing work to provide SIMD support for commonly used functions (the version Numpy 1.22.0rc1 not yet released continue this long-standing work). Moreover, the compiler or the processor used may significantly impact the results. The following experiments have been done using a Numpy retrieved from pip on a Debian Linux with a i5-9600KF processor.

Under the hood of `np.sum`

For floating-point numbers, Numpy uses a *pairwise algorithm* which is known to be quite *numerically stable* while being relatively fast. This can be seen in the code, but also simply using a profiler: `TYPE_pairwise_sum` is the C function called to compute the sum at runtime (where `TYPE` is `DOUBLE` or `FLOAT`).

For integers, Numpy use a classical naive reduction. The C function called is `ULONG_add_avx2` on AVX2-compatible machines. It also surprisingly convert items to 64-bit ones if the type is not `np.int64` .

Here is the hot part of the assembly code executed by the `DOUBLE_pairwise_sum` function

```
 3,65 | a0:┌─→add        $0x8,%rcx                  ; Loop iterator
 3,60 |    │  prefetcht0 (%r8,%rax,1)               ; Prefetch data
 9,46 |    │  vaddsd     (%rax,%rbp,1),%xmm1,%xmm1  ; Accumulate an item in xmm1[0]
 4,65 |    │  vaddsd     (%rax),%xmm0,%xmm0         ; Same for xmm0[0]
 6,91 |    │  vaddsd     (%rax,%rdx,1),%xmm4,%xmm4  ; Etc.
 7,77 |    │  vaddsd     (%rax,%rdx,2),%xmm7,%xmm7
 7,41 |    │  vaddsd     (%rax,%r10,1),%xmm2,%xmm2
 7,27 |    │  vaddsd     (%rax,%rdx,4),%xmm6,%xmm6
 6,80 |    │  vaddsd     (%rax,%r11,1),%xmm3,%xmm3
 7,46 |    │  vaddsd     (%rax,%rbx,1),%xmm5,%xmm5
 3,46 |    │  add        %r12,%rax                  ; Switch to the next items (x8)
 0,13 |    ├─cmp        %rcx,%r9                   ; Should the loop continue?
 3,27 |    └─jg         a0                         ; Jump to the beginning if so
```

The Numpy compiled code clearly uses the **scalar SIMD instruction** `vaddsd` (computing only a single double-precision item) although it successfully *unroll the loop* 8 times. The same code is generated for `FLOAT_pairwise_sum` : `vaddss` is called 8 times.

For the `np.uint32` , here is the hot part of the generated assembly code:

```
 2,37 |160:┌─→add          $0x1,%rax      ; Loop iterator
95,95 |    │  add          (%rdi),%rdx    ; Accumulate the values in %rdx
 0,06 |    │  add          %r10,%rdi      ; Switch to the next item
      |    ├─cmp          %rsi,%rax      ; Should the loop continue?
 1,08 |    └─jne          160            ; Jump to the beginning if so
```

Numpy obviously does not use SIMD instructions for `np.uint32` type. It does not even unroll the loop. The `add (%rdi),%rdx`

Join Stack Overflow to find the best answer to your technical question, help others answer theirs.

Sign up with email

However, the `np.uint32` version call the C function `_aligned_contig_cast_uint_to_ulong` in order to **convert the integer items to a wider type**. Numpy does that to **avoid integer overflows**. The same thing can be seen for the types `np.uint8` and `np.uint16` (although the name of the function differs). Hopefully, this function makes use of SIMD instructions (SSE) but still takes a significant portion of the execution time (~30% of the `np.sum` time).

EDIT: as pointed out by @user2357112supportsMonica, the `dtype` parameter of `np.sum` can be explicitly specified. When it match with the `dtype` of the input array, then the conversion is not performed. This results in a smaller execution time at the expense of a possible overflow.

Benchmark results

Here is the result on my machine:

```
uint16: 7.17 µs ± 80 ns per loop (mean ± std. dev. of 7 runs, 20000 loops each)
uint32: 7.11 µs ± 12.3 ns per loop (mean ± std. dev. of 7 runs, 20000 loops each)
uint64: 5.05 µs ± 8.57 ns per loop (mean ± std. dev. of 7 runs, 20000 loops each)
float32: 2.88 µs ± 9.27 ns per loop (mean ± std. dev. of 7 runs, 20000 loops each)
float64: 3.06 µs ± 10.6 ns per loop (mean ± std. dev. of 7 runs, 20000 loops each)
```

First of all, not that the results are very similar to the ones provided in the question meaning that the behavior seen on my machine can be successfully reproduced on other one. Thus, the explanation should also be consistent.

As you can see, the 64-bit version is faster than the other integer-based versions. This is due to the overhead of the conversion. The two first are equally+fast because of the scalar loop and the `add` instruction being equally-fast for 8-bit, 16-bit and 32-bit integers (this should be true for most 64-bit mainstream platforms). The integer implementations are slower than the floating-point ones because of the lack of (proper) loop unrolling.

The floating-point implementations are equally-fast due to the scalar SIMD instructions. Indeed, the instructions `vaddss` (for `np.float32`) and `vaddsd` (for `np.float64`) have the same latency and throughput (at least on all modern Intel processors). Thus the throughput of the two implementation is the same since the loop of the two implementation is similar (same unrolling).

Additional notes

This is sad `np.sum` does not fully make use of SIMD instructions as this would speed up a lot the computations using it (especially small integers).

[UPDATE] Looking at the Numpy code, I discovered that the code is not vectorized because the **array stride is a runtime value** and the compiler do not generate a specialized version where the stride is 1. In fact, this can be partially seen in the previous assembly code: the compiler used the instruction `add %r10, %rdi` because `%r10` (the stride of the target array) is not known at compile-time. There is currently no optimization for this specific case of reduction in the Numpy code yet (the functions are relatively generic). This may change in a near future.

In addition to the stride problem, one big point makes it hard for compilers to automatically vectorize a code: the floating-point addition is not commutative, nor associative (unless flags like `-ffast-math` are used).

Share Improve this answer Follow

edited Nov 29, 2021 at 0:22

answered Nov 27, 2021 at 16:00

Jérôme Richard
42.9k 6 30 62

2 Oh, right. I totally forgot about the weird default dtype behavior `numpy.sum` and `numpy.ndarray.sum` have. What do you see if you explicitly do something like `int16.sum(dtype='uint16')` ? – user2357112 Nov 27, 2021 at 18:59

Join Stack Overflow to find the best answer to your technical question, help others answer theirs.

Sign up with email

2 SIMD summation without integer overflow isn't hard, you just unpack with zeros. All mainstream SIMD ISAs have shuffles that make it possible to do that. For integer stuff it's easy to get identical results even with different vectorization strategies (because there's never any rounding, or if you do it right no overflow of any accumulators narrower than the final result), so you can manually vectorize for some common ISAs but not others without any observable change in behaviour except performance, so the non-portability SIMD isn't much of an excuse. (It's baseline for x86-64 and ARM64.) – Peter Cordes Nov 27, 2021 at 22:03

1 (There are some per-architecture tricks like x86 using `pmaddwd` with `_mm_set1_epi16(1)` to horizontal sum pairs of int16 into int32 more efficiently than unpacking low/high with zeros for unsigned or SSE4.1 `pmovsxwd` for signed, and only widening to 64-bit elements in an outer loop after some number of iterations that guarantees 32-bit sums of 16-bit elements won't overflow. Or to sum bytes without overflow, x86 `psadbw` against `_mm_setzero_si128()` . See Fastest way to do horizontal SSE vector sum but just widen elements, not all the way to scalar) – Peter Cordes Nov 27, 2021 at 22:07

1 @JérômeRichard Thanks a lot for the detailed answer! This thread has been a great discussion all-around and I've learned a lot! I'll try to see if I can modify my code to use `sum(dtype=...)` explicitly – pnjun Nov 28, 2021 at 13:26

Join Stack Overflow to find the best answer to your technical question, help others answer theirs.

Sign up with email